

line. Dead code is misleading because programmers reading it assume that it's an active part of the program when it's effectively the same as commented-out code.

Stubs are an exception to these code smell rules. These are placeholders for future code, such as functions or classes that have yet to be implemented. In lieu of real code, a stub contains a `pass` statement, which does nothing. (It's also called a *no operation* or *no-op*.) The `pass` statement exists so you can create stubs in places where the language syntax requires some code:

```
>>> def exampleFunction():  
...     pass  
...
```

When this function is called, it does nothing. Instead, it indicates that code will eventually be added in.

Alternatively, to avoid accidentally calling an unimplemented function, you can stub it with a `raise NotImplementedError` statement. This will immediately indicate that the function isn't yet ready to be called:

```
>>> def exampleFunction():  
...     raise NotImplementedError  
...  
>>> exampleFunction()  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
  File "<stdin>", line 2, in exampleFunction  
NotImplementedError
```

Raising a `NotImplementedError` will warn you whenever your program calls a stub function or method by accident.

Commented-out code and dead code are code smells because they can mislead programmers into thinking that the code is an executable part of the program. Instead, remove them and use a version control system, such as Git or Subversion, to keep track of changes. Version control is covered in Chapter 12. With version control, you can remove the code from your program and, if needed, easily add it back in later.